



Árvores de Busca Binária e AVL



Prof. Rafael Mesquita
rgm@cin.ufpe.br

O que veremos nesta aula?

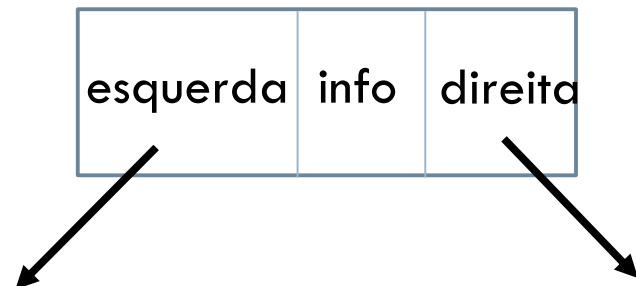
- Árvores de busca binária (BST)
- Árvores AVL
- Execício

Motivação

- Árvores BST ou de pesquisa binária
 - ▣ Estruturas de dados que podem ser utilizados como um dicionário ou fila de prioridades
 - ▣ Operações básicas possuem complexidade proporcional à altura da árvore ($O(h)$)
 - ▣ Para árvores completas
 - Princípio da busca binária
 - Busca, inserção e deleção
 - $\Theta(\log(n))$
 - Á cada comparação em um nó visitado, aproximadamente metade da subárvore restante é descartada

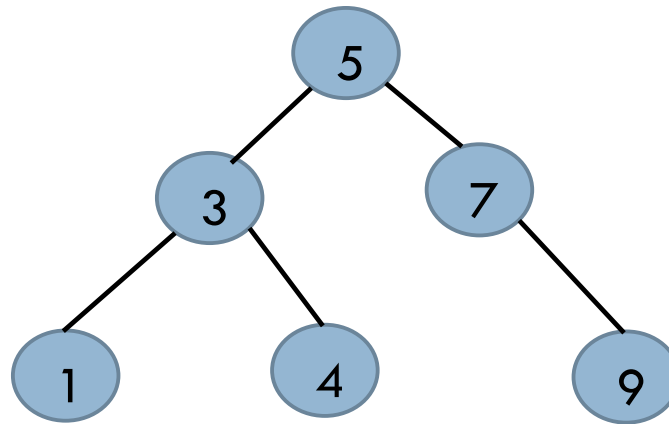
Árvore de Pesquisa Binária

- Organizada como uma árvore binária
- Além disso:
 - ▣ Dado um nó x de uma BST
 - ▣ Para todo nó y da subárvore à esquerda de x
 - $y \rightarrow info \leq x \rightarrow info$
 - ▣ Para todo nó y da subárvore à direita de x
 - $y \rightarrow info > x \rightarrow info$



Árvore de Pesquisa Binária

□ Exemplo:



▣ Percurso em-ordem imprime os elementos da árvore em uma sequência ordenada de forma crescente

Árvore de Pesquisa Binária

□ Consulta

- ▣ Procura por um determinado elemento na árvore

▣ Variações

- Busca por máximo, mínimo, antecessor e sucessor
- Todas ocorrem com complexidade proporcional à altura da árvore
- ▣ Caso elemento da raiz seja maior que o elemento buscado
 - Busca na subárvore à esquerda
- ▣ Caso elemento da raiz seja menor que o elemento buscado
 - Busca na subárvore à direita

Árvore de Pesquisa Binária

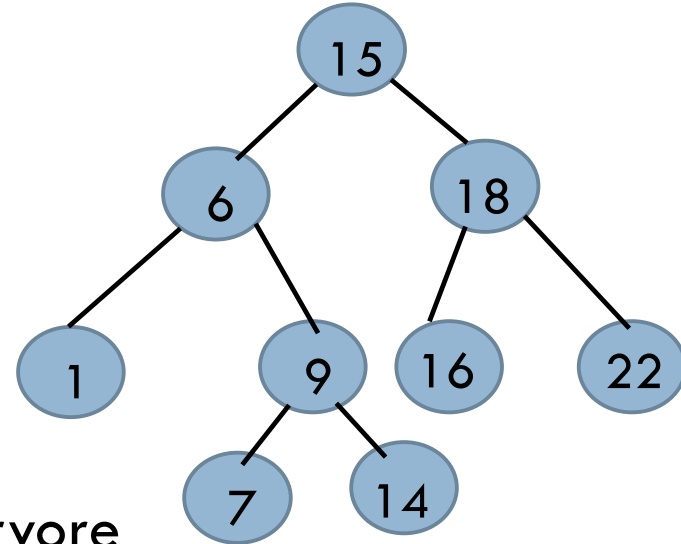
□ Consulta

□ Busca se encerra quando

- Árvore vazia for visitada
 - Elemento não pertence à árvore
- Elemento for encontrado na raiz

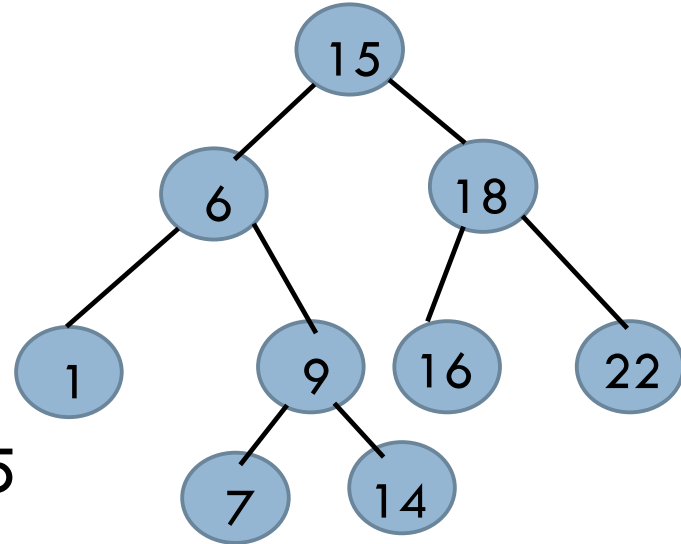
Árvore de Pesquisa Binária

- Consulta
- Busca por 9:
 - ▣ $15 \rightarrow 6 \rightarrow 9$
- Busca por 17:
 - ▣ $15 \rightarrow 18 \rightarrow 16 \rightarrow \text{NULL}$
 - ▣ Elemento não pertence à Árvore
- Mínimo: 1
 - ▣ Acessa sempre a subárvore à esquerda
- Máximo: 22
 - ▣ Acessa sempre a subárvore à direita



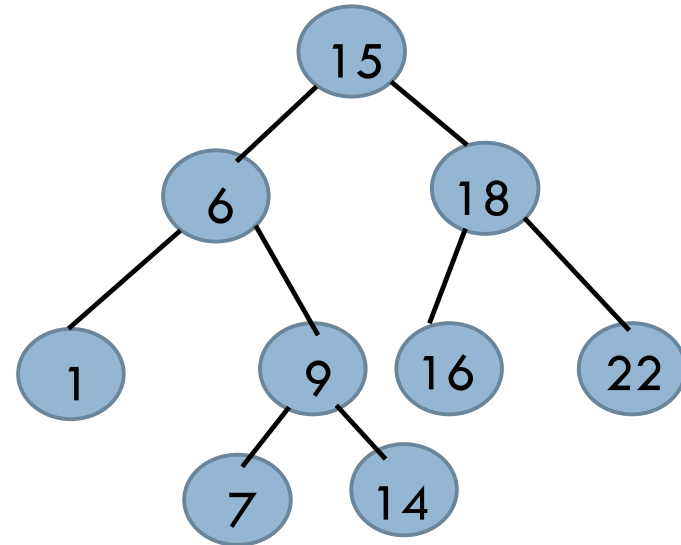
Árvore de Pesquisa Binária

- Consulta
- Sucessor de 15
 - ▣ 16
 - ▣ Elemento mínimo da subárvore à direita de 15
- Antecessor de 15
 - ▣ 14
 - ▣ Elemento máximo da subárvore à esquerda de 15



Árvore de Pesquisa Binária

- Consulta
- Sucessor de 14
 - ▣ 15
- Sucessor de 16
 - ▣ 18
- Como subarvore à direita é nula
 - ▣ Sucessor é o ancestral mais alto de valor maior que o nó em questão
 - ▣ Sucessor é o ancestral mais alto, cujo filho à esquerda também é um ancestral, ou o próprio nó



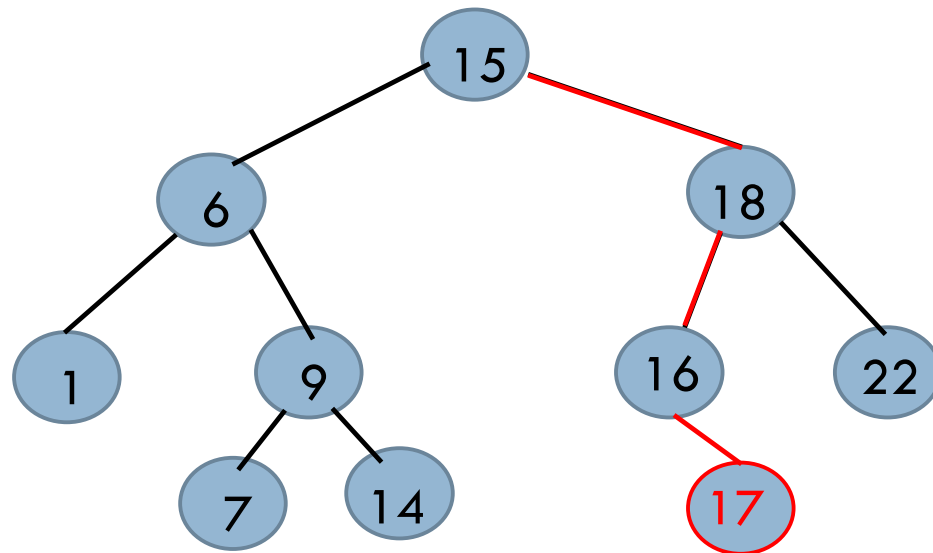
Árvore de Pesquisa Binária

- Inserção e eliminação
 - ▣ Modifica o conjunto de elementos representado pela Árvore
 - ▣ Propriedade de árvore de busca binária deve continuar válida
 - Elementos menores na subárvore à esquerda
 - Elementos maiores na subárvore à direita

Árvore de Pesquisa Binária

□ Inserção

- Busca da posição de inserção começa na raiz até a subárvore vazia à esquerda ou à direita de uma folha
- Ex: inserindo 17



Árvore de Pesquisa Binária

```
void insereInfo(Arvore* raiz, int valor)
{
    Arvore* novoNo = cria(valor, NULL, NULL);
    inserePosicao(raiz, novoNo);
}

void inserePosicao(Arvore* noAtual, Arvore* novoNo)
{
    if (noAtual == NULL || novoNo == NULL)
        return;
    if (novoNo->info > noAtual->info)
    {
        if (noAtual->direita == NULL)
        {
            noAtual->direita = novoNo;
            return;
        }
        else
        {
            inserePosicao(noAtual->direita, novoNo);
        }
    }
}
```

Árvore de Pesquisa Binária

```
else {  
    if (noAtual->esquerda == NULL)  
    {  
        noAtual->esquerda = novoNo;  
        return;  
    }  
    else  
    {  
        inserePosicao(noAtual->esquerda, novoNo);  
    }  
}
```

Árvore de Pesquisa Binária

□ Eliminação

- ▣ Considere um nó Z a ser removido

- ▣ 3 casos possíveis

1. Z não possui nenhum filho;

1. Pai de Z modifica ponteiro que apontava para Z para NULL

2. Z possui apenas um filho;

1. Ponteiro do pai de Z que apontava para Z passa apontar para o filho de Z

3. Z possui dois filhos

1. Extraímos o sucessor de Z , y , que possui, no máximo, um filho, e substituímos Z por y

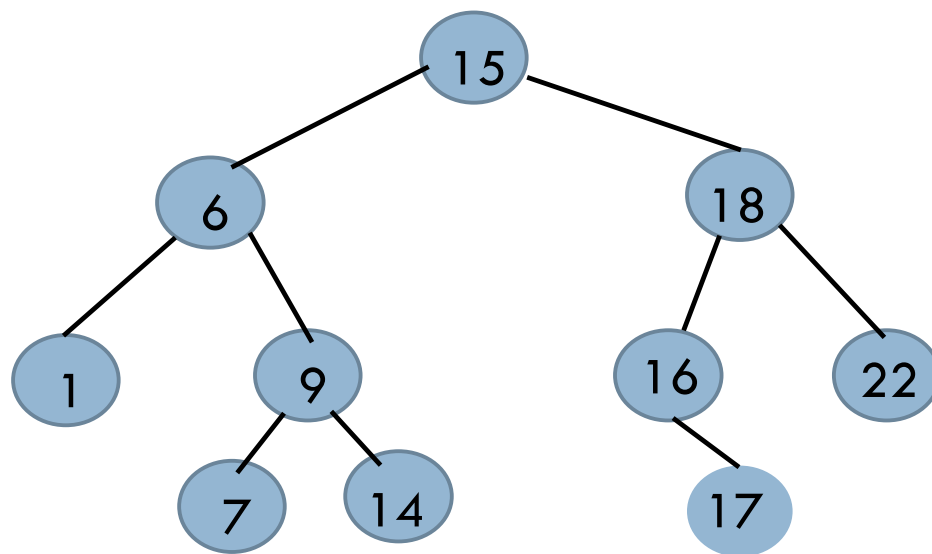
2. Removemos Z

Árvore de Pesquisa Binária

□ Eliminação

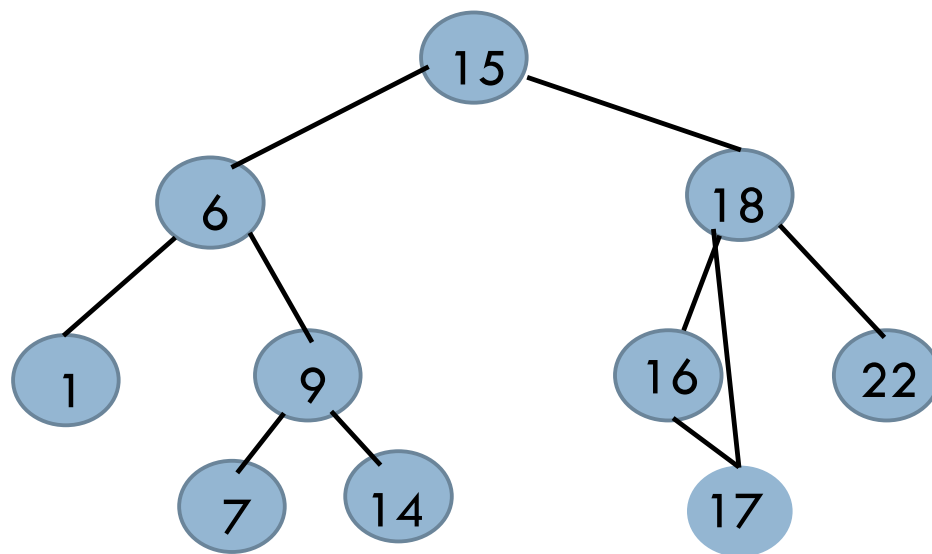
- Primeiro caso

- Ex: removendo 14



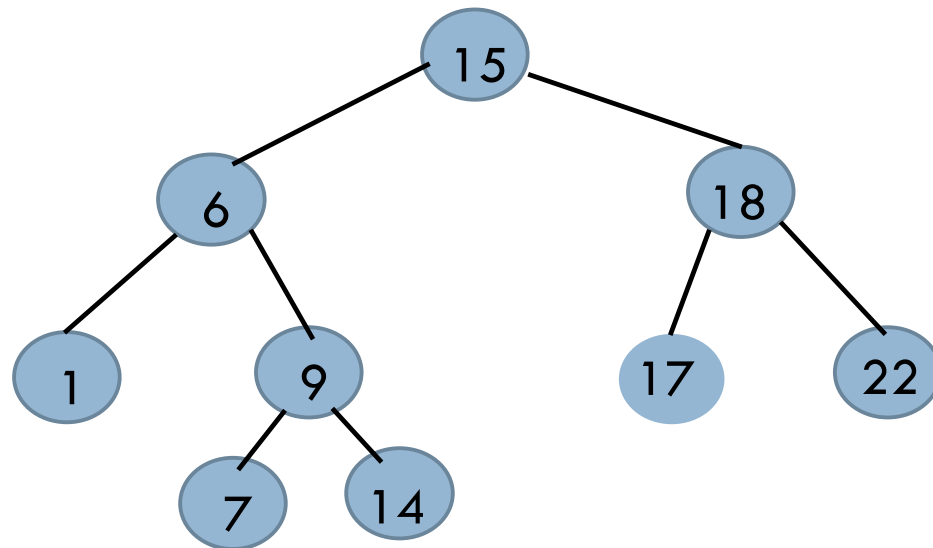
Árvore de Pesquisa Binária

- Eliminação
 - Segundo caso
 - Ex: removendo 16



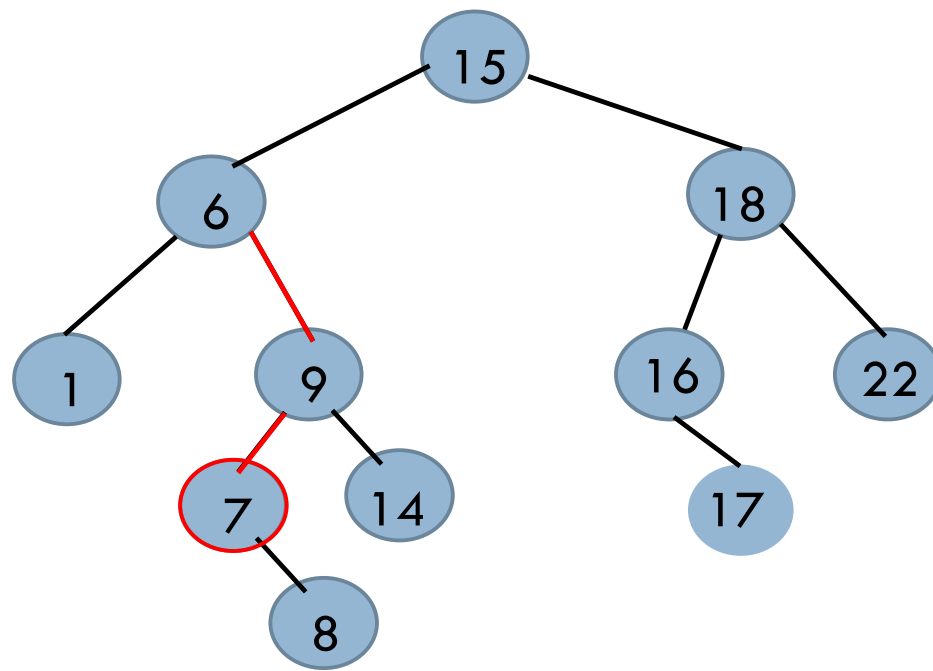
Árvore de Pesquisa Binária

- Eliminação
 - Segundo caso
 - Ex: removendo 16



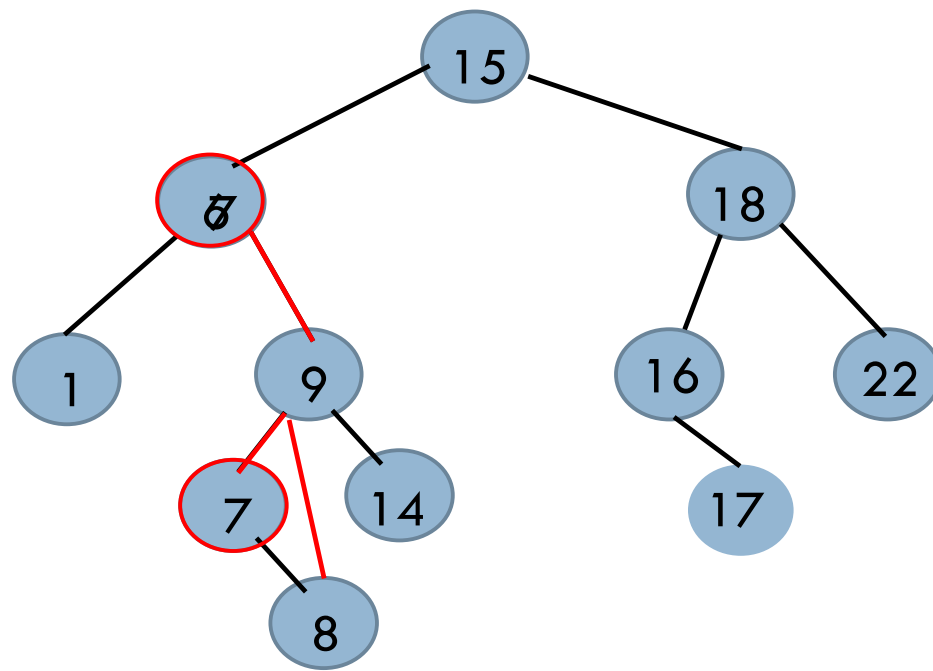
Árvore de Pesquisa Binária

- Eliminação
 - Terceiro caso
 - Ex: removendo 6



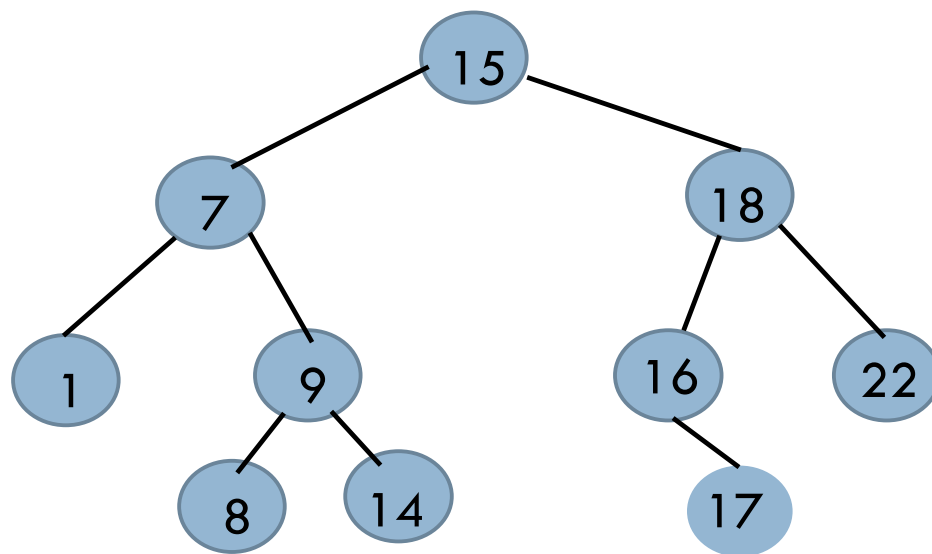
Árvore de Pesquisa Binária

- Eliminação
 - Terceiro caso
 - Ex: removendo 6



Árvore de Pesquisa Binária

- Eliminação
 - Terceiro caso
 - Ex: removendo 6



Árvore de Pesquisa Binária

```
void deleta(Arvore** noAtual, int valor)
{
    if ((*noAtual) == NULL)
        return;

    if ((*noAtual)->info == valor)
    {
        if ((*noAtual)->direita == NULL && (*noAtual)->esquerda == NULL)
        {
            free(*noAtual);
            *noAtual = NULL;
        }
        else if ((*noAtual)->direita == NULL)
        {
            Arvore* aux;
            aux = *noAtual;
            *noAtual = (*noAtual)->esquerda;
            free(aux);
        }
    }
}
```

Árvore de Pesquisa Binária

```
else if ((*noAtual)->esquerda == NULL)
{
    Arvore* aux;
    aux = *noAtual;
    *noAtual = (*noAtual)->direita;
    free(aux);
}
```

Árvore de Pesquisa Binária

```
else {
    Arvore* sucessor = (*noAtual)->direita;
    Arvore* noPaiSucessor = (*noAtual);

    if (sucessor->esquerda == NULL)
    {
        noPaiSucessor->direita = sucessor->direita;
    }
    else {
        while (sucessor->esquerda != NULL)
        {
            noPaiSucessor = sucessor;
            sucessor = sucessor->esquerda;
        }
        noPaiSucessor->esquerda = sucessor->direita;
    }
    (*noAtual)->info = sucessor->info;
    free(sucessor);
}
} //fecha if ((*noAtual)->info == valor)
```

Árvore de Pesquisa Binária

```
else if (valor > (*noAtual)->info)
{
    deleta(&(*noAtual)->direita, valor);
}
else {
    deleta(&(*noAtual)->esquerda, valor);
}
} //fecha funcao deleta()
```

Árvore de Pesquisa Binária

```
int main()
{
    Arvore* raiz = cria(7, NULL,
    NULL);
    insereInfo(raiz, 5);
    insereInfo(raiz, 13);
    insereInfo(raiz, 3);
    insereInfo(raiz, 6);
    insereInfo(raiz, 1);
    insereInfo(raiz, 4);
    insereInfo(raiz, 10);
    insereInfo(raiz, 12);
    insereInfo(raiz, 8);
    insereInfo(raiz, 9);
    insereInfo(raiz, 17);
    insereInfo(raiz, 16);

    insereInfo(raiz, 14);
    insereInfo(raiz, 15);
    insereInfo(raiz, 20);
    deleta(&raiz, 7);
    pre_ordem(raiz);
    Arvore* noBuscado = busca(raiz, 13);
    if (noBuscado != NULL)
        printf("\n\n%d", noBuscado->info);
    else
        printf("\n\nnao encontrado!");
    printf("\n\n%d", raiz->info);
    getchar();
    return 0;
}
```

Árvore AVL

- Georgy Adelson-Velsky and Evgenii Landis, 1962
 - Garantem complexidade de $O(\log(n))$ no pior caso das operações de busca, inserção e remoção
 - Necessário garantir que a árvore permanece balanceada após inserções e remoções
 - Balanceamento
 - Altura da árvore limitada à $O(\log(n))$
 - $n = \text{número de nós}$

Árvore AVL

□ Definição:

- É uma árvore de busca binária
- Para cada nó, a diferença entre as alturas das subárvores à esquerda e à direita é, no máximo, 1
 - Busca requer, no máximo, $\log(n)$ comparações
- Custo: inserções e remoções devem garantir essa propriedade

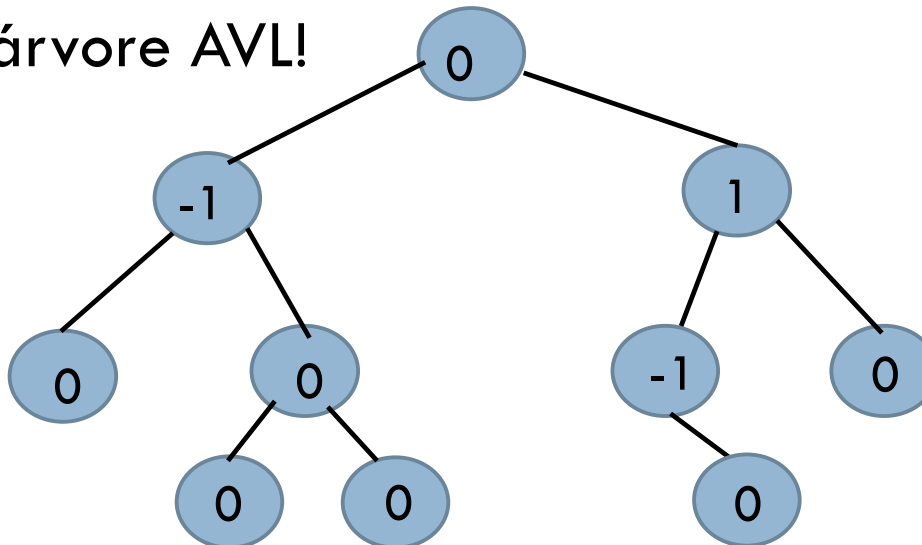
Árvore AVL

□ Definição:

- Em cada nó da árvore é especificada a diferença entre as alturas das subárvores da esquerda e da direita (*fator de balanceamento* – *fb*)

- Nó armazena valor de *fb*

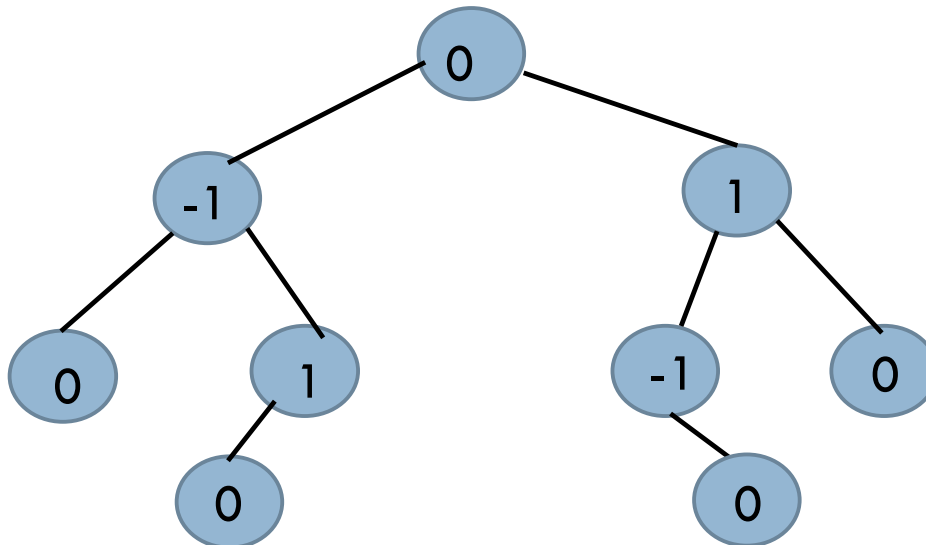
- É uma árvore AVL!



Árvore AVL

□ Definição:

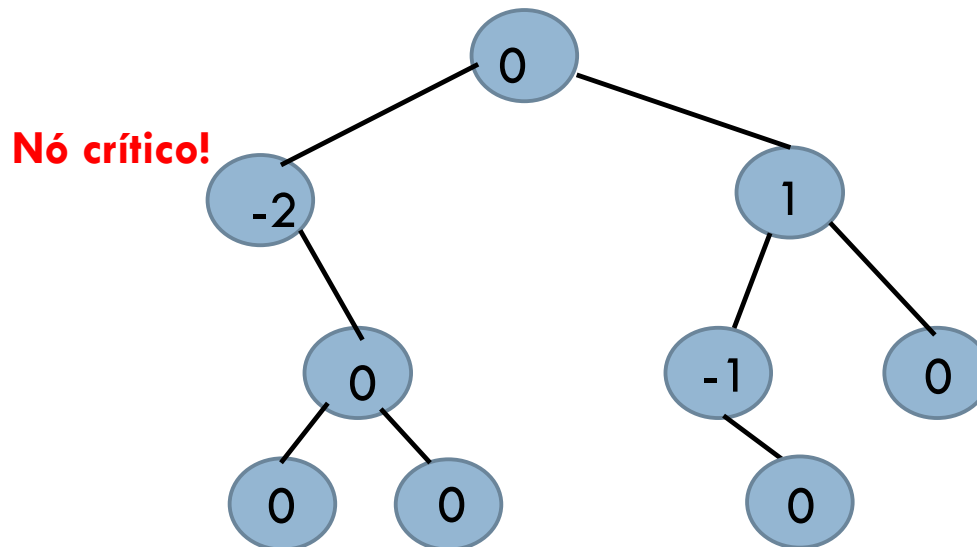
- Em cada nó da árvore é especificada a diferença entre as alturas das subárvores da esquerda e da direita (*fator de balanceamento* – *fb*)
- É uma árvore AVL!



Árvore AVL

□ Definição:

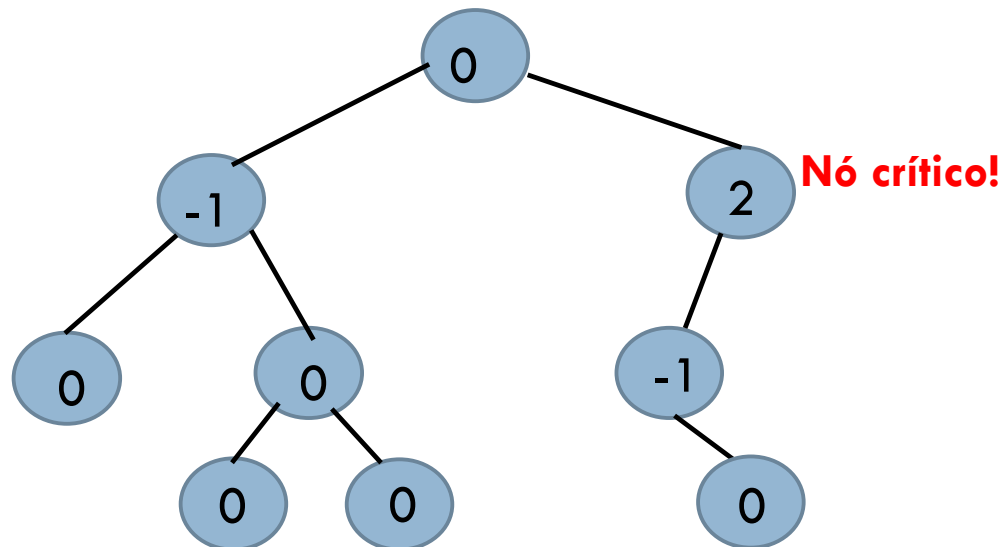
- Em cada nó da árvore é especificada a diferença entre as alturas das subárvores da esquerda e da direita (*fator de balanceamento* – *fb*)
- **Não** é uma árvore AVL!



Árvore AVL

□ Definição:

- Em cada nó da árvore é especificada a diferença entre as alturas das subárvores da esquerda e da direita (*fator de balanceamento* – *fb*)
- **Não** é uma árvore AVL!



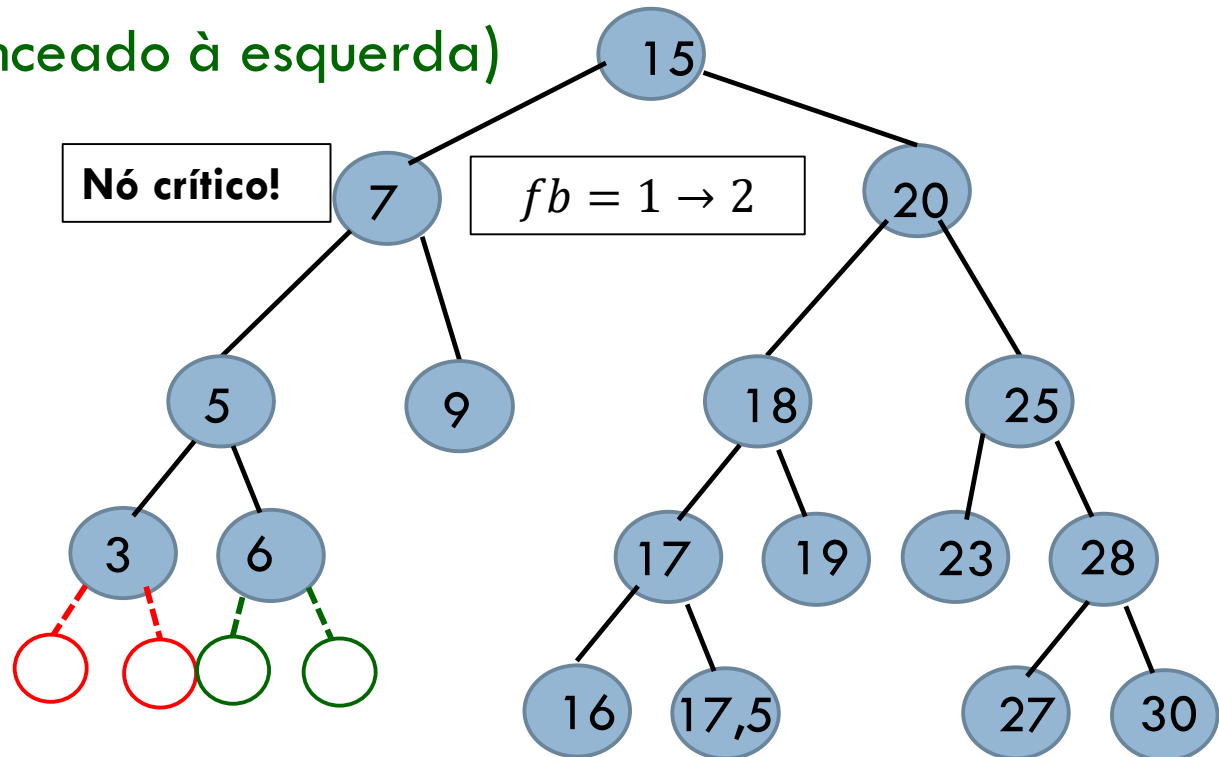
Árvore AVL

□ Inserção

- ▣ Inserção é realizada do modo usual para uma BST
- ▣ Caso após a inserção, a árvore continue sendo AVL, nada mais precisa ser feito
- ▣ Do contrário, a árvore precisa ser rebalanceada
 - Operação de **rotação**!

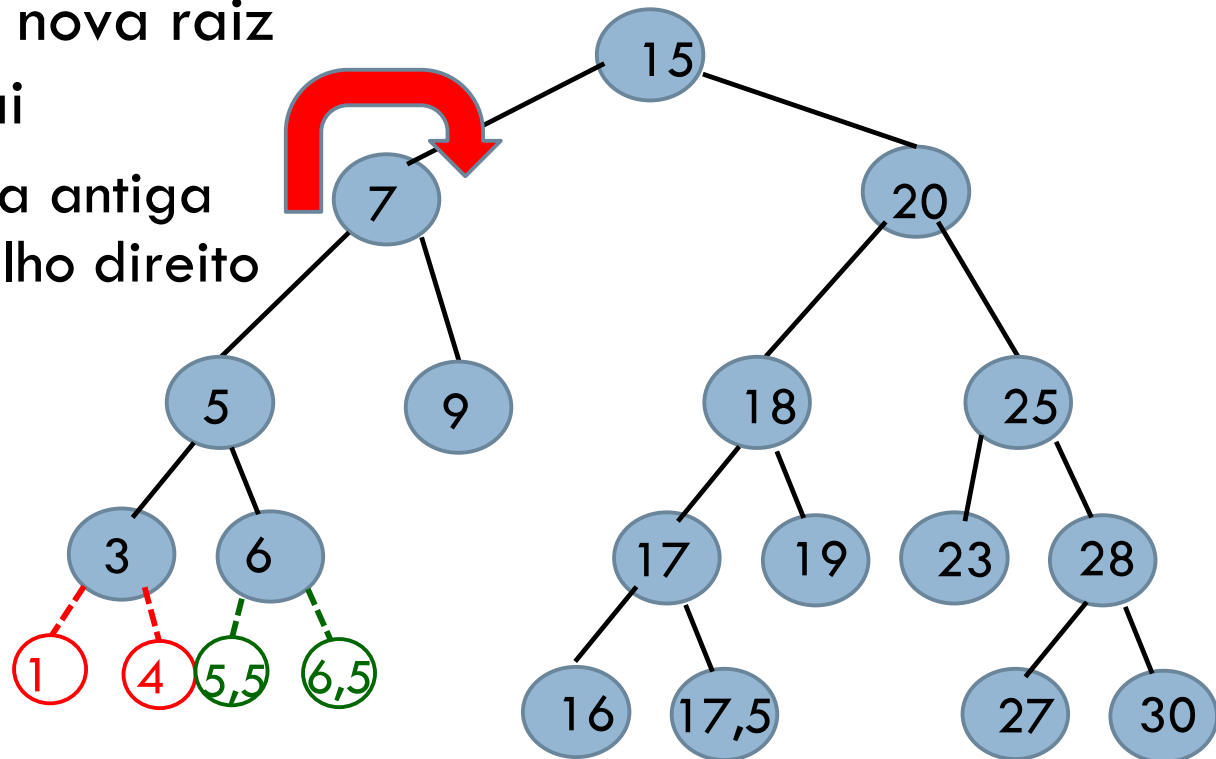
Árvore AVL

- Considere a inserção de um dos nós em verde ou vermelho
- Inserção em subárvore à esquerda de descendente à esquerda de um nó com $fb = 1$ (balanceado à esquerda)
- Inserção em subárvore à direita de descendente à esquerda de um nó com $fb = 1$ (balanceado à esquerda)



Árvore AVL

- Rotação simples à direita
 - ▣ Filho esquerdo do nó crítico vira raiz
 - ▣ Filho esquerdo da nova raiz é mantido
 - ▣ Filho direito da nova raiz é seu antigo pai
 - ▣ Filho esquerdo da antiga raiz é o antigo filho direito da nova raiz

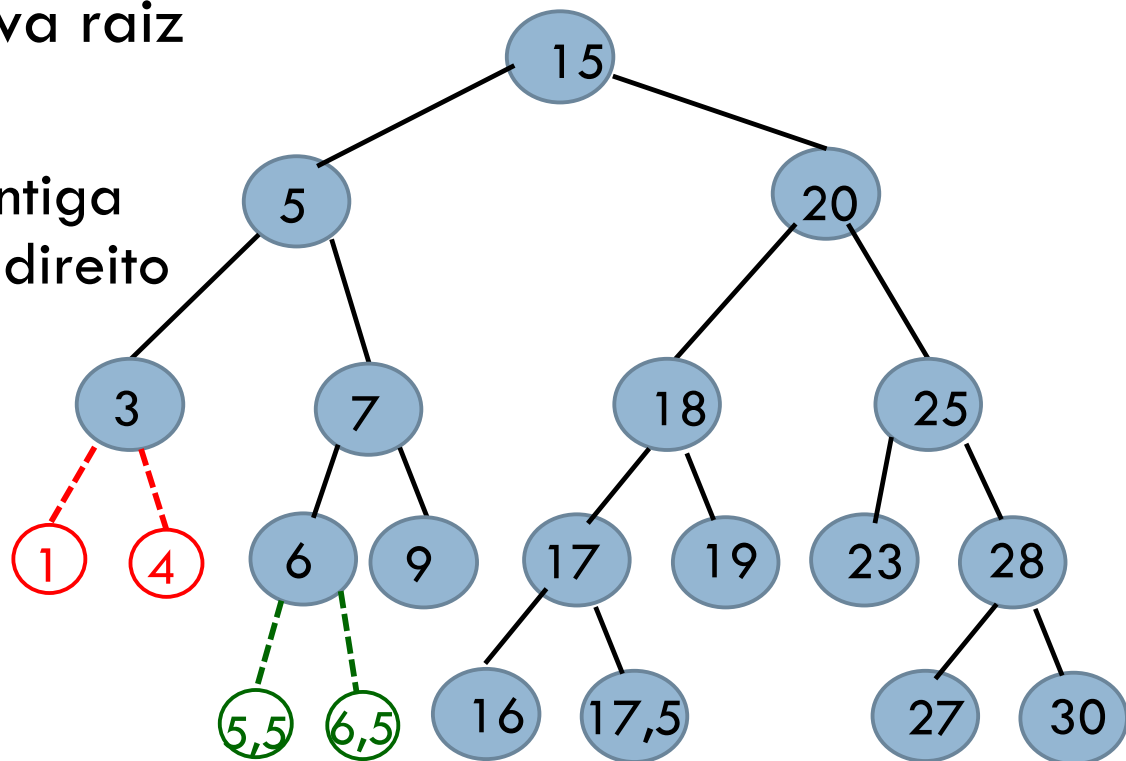


Árvore AVL

- Rotação simples à direita
 - ▣ Filho esquerdo do nó crítico vira raiz
 - ▣ Filho esquerdo da nova raiz é mantido
 - ▣ Filho direito da nova raiz é seu antigo pai
 - ▣ Filho esquerdo da antiga raiz é o antigo filho direito da nova raiz

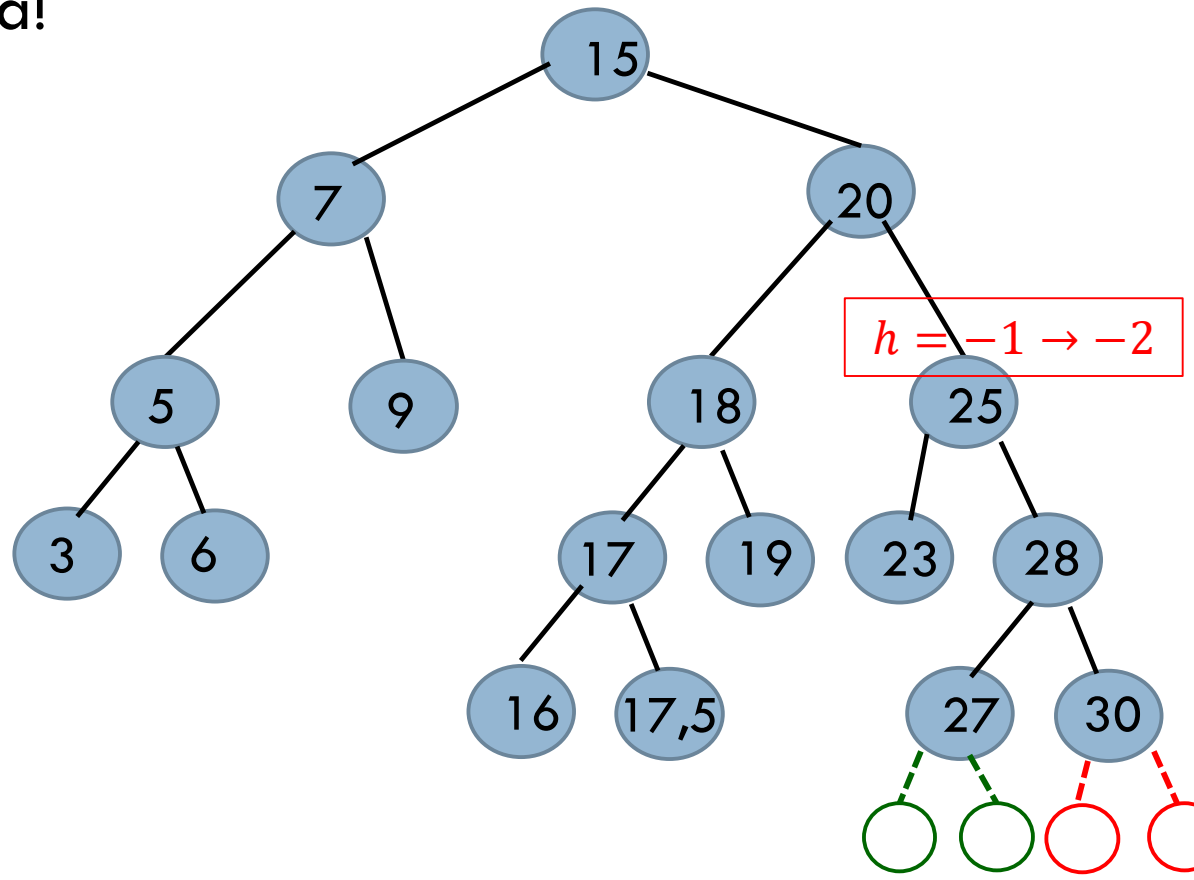
Árvore continua sendo AVL e, consequentemente, BST!

Nó 5 desbalanceado ($dif = -2$).
Árvore não é AVL!



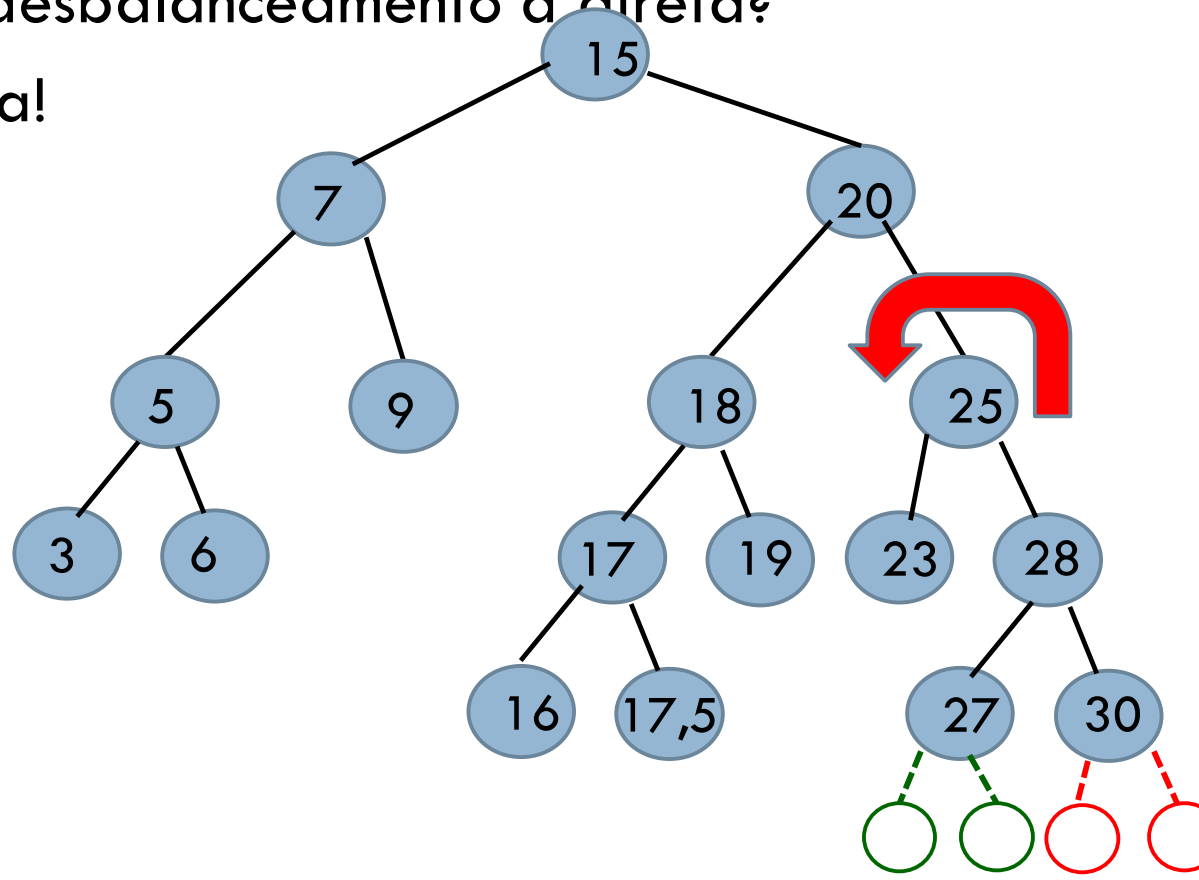
Árvore AVL

- Tipos de desbalanceamento
 - ▣ Como resolver um desbalanceamento à direita?
 - ▣ Rotação à esquerda!



Árvore AVL

- Considere a inserção de um dos nós em verde ou vermelho
- Como resolver um desbalanceamento à direita?
- Rotação à esquerda!



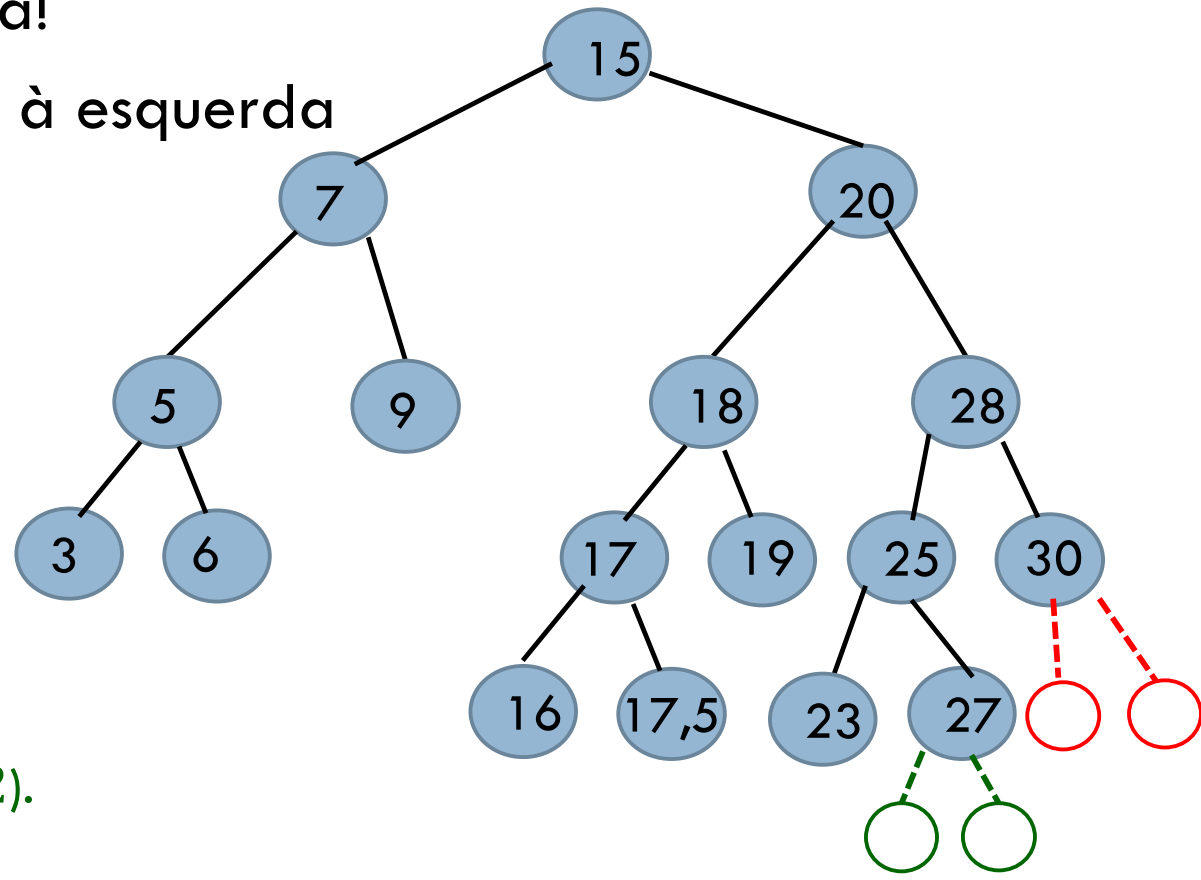
Árvore AVL

□ Tipos de desbalanceamento

▣ Como resolver um desbalanceamento à direita?

▣ Rotação à esquerda!

▣ Ex: rotação simples à esquerda



Árvore continua sendo AVL e,
consequentemente, BST!

Nó 28 desbalanceado ($dif = 2$).

Árvore não é AVL!

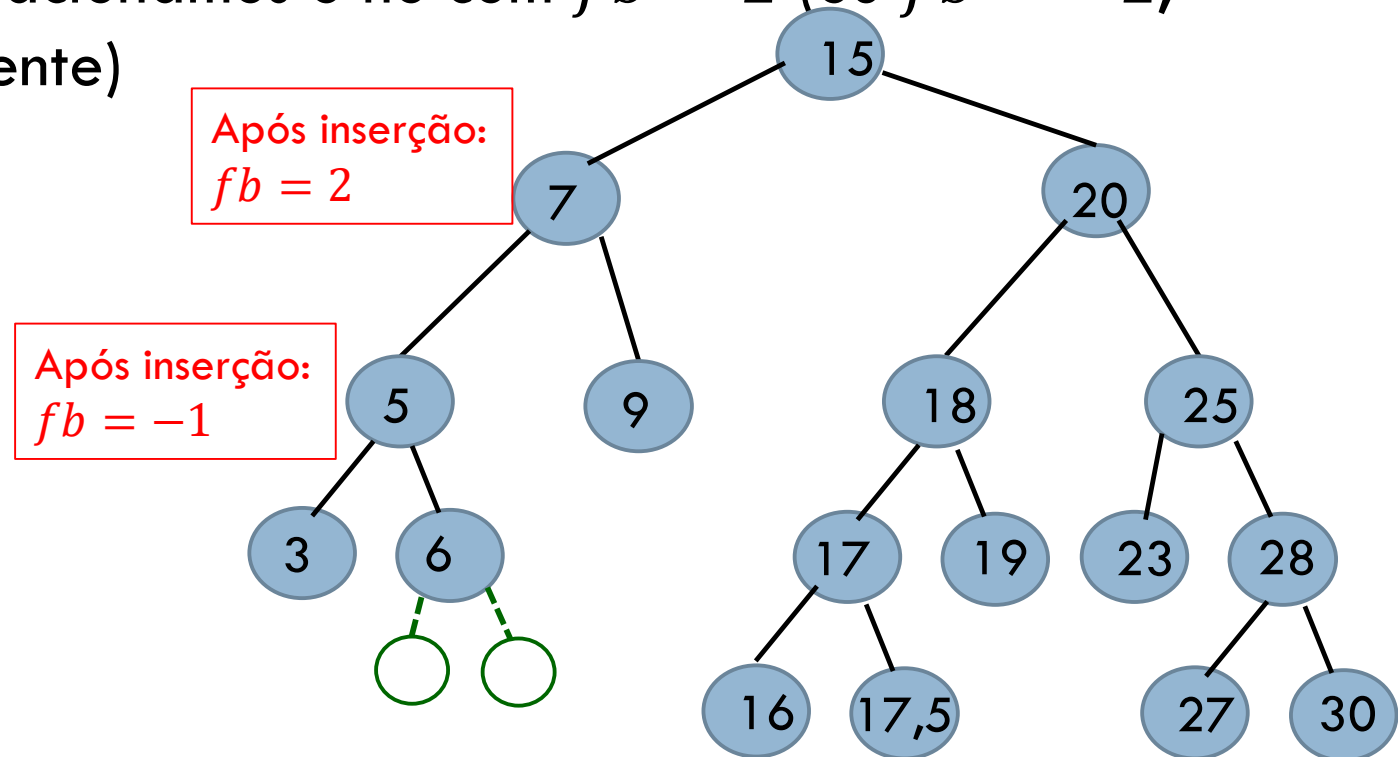
Árvore AVL

- Tipos de balanceamento
- Inserção em subárvore à esquerda de descendente à esquerda de um nó com $dif = 1$ (balanceado à esquerda)
 - ▣ Rotação simples funciona!
- Inserção em subárvore à direita de descendente à direita de um nó com $dif = -1$ (balanceado à direita)
 - ▣ Rotação simples funciona!
- Inserção em subárvore à direita de descendente à esquerda de um nó com $dif = 1$ (balanceado à esquerda)
 - ▣ Rotação simples não funciona! Necessária rotação dupla!
- Inserção em subárvore à esquerda de descendente à direita de um nó com $dif = -1$ (balanceado à direita)
 - ▣ Rotação simples não funciona! Necessária rotação dupla!

Árvore AVL

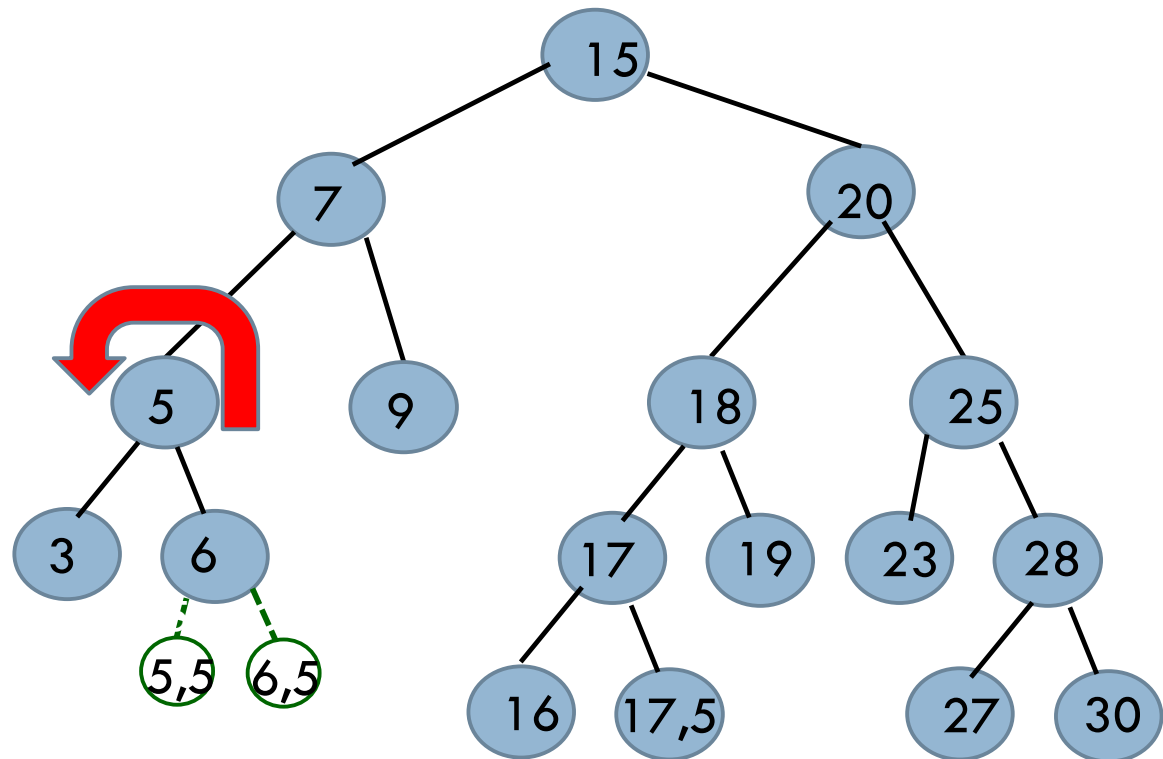
□ Rotação dupla

- ▣ Primeiro, rotacionamos o nó com $fb = -1$ (ou $fb = 1$, analogamente)
- ▣ Depois, rotacionamos o nó com $fb = 2$ (ou $fb = -2$, analogamente)



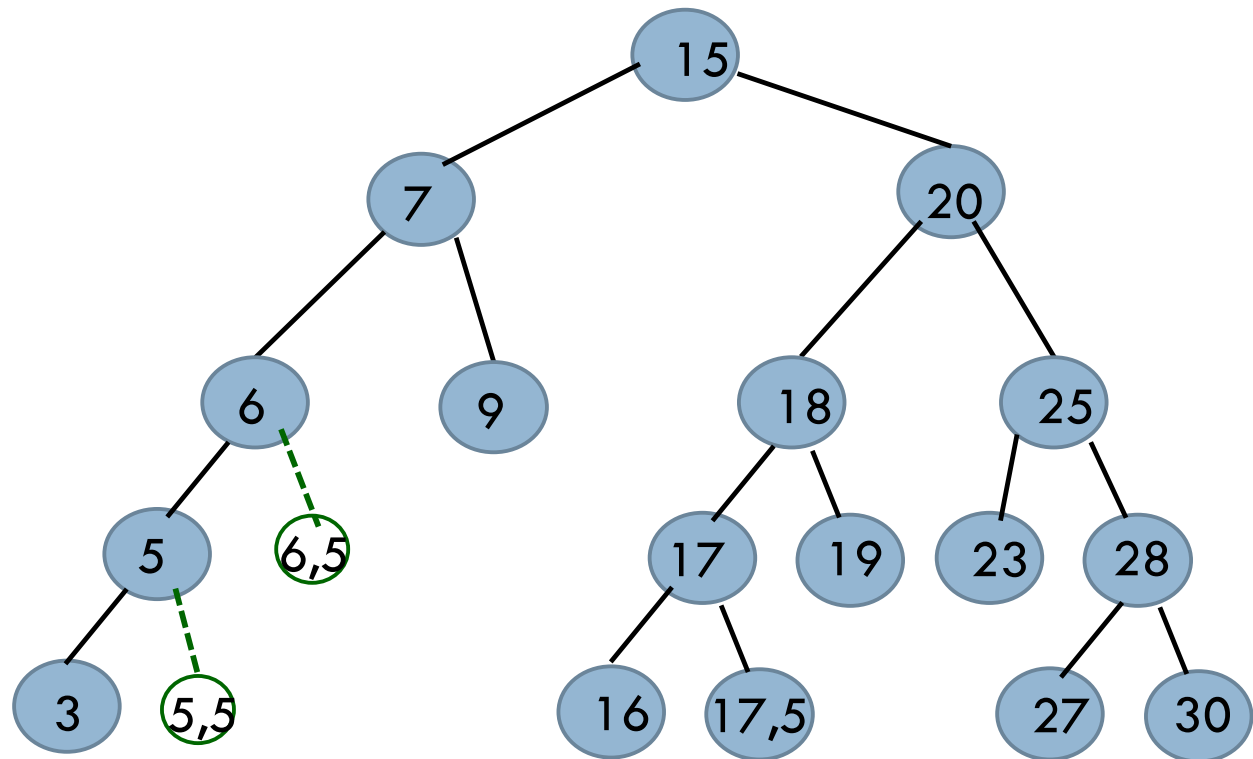
Árvore AVL

□ Rotação dupla



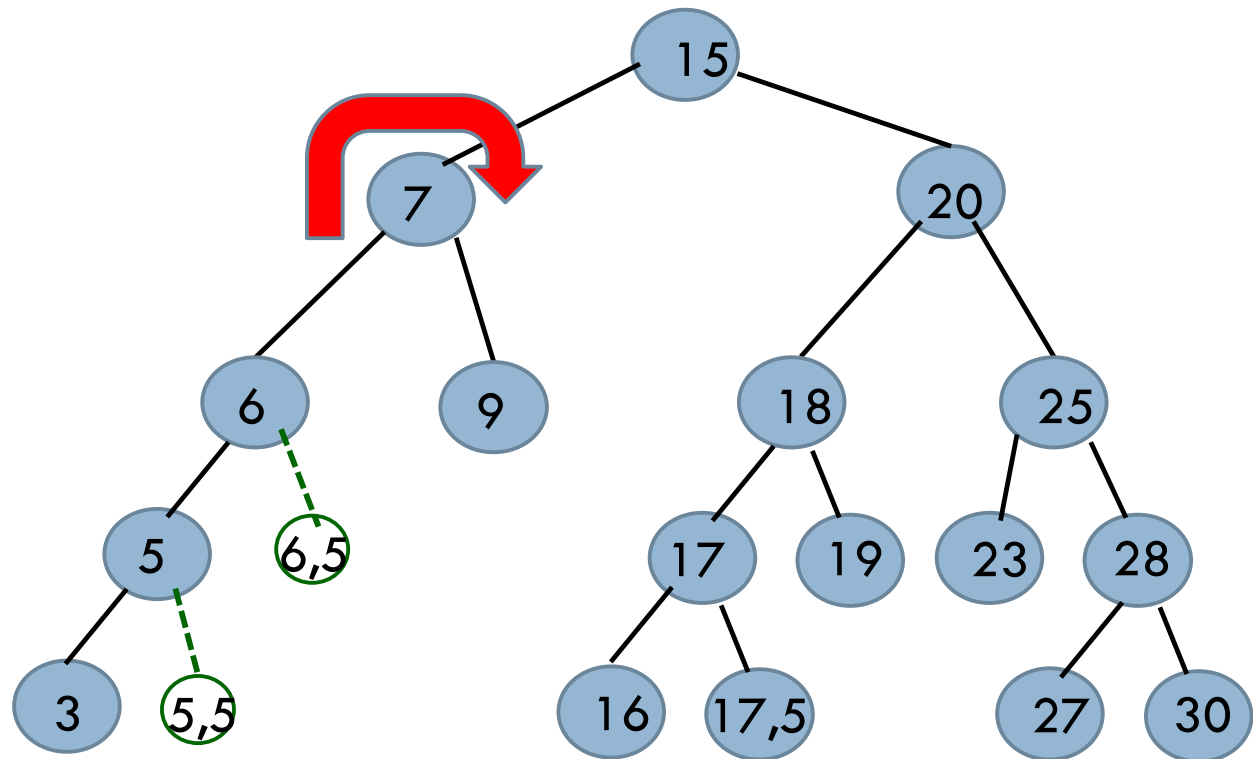
Árvore AVL

□ Rotação dupla



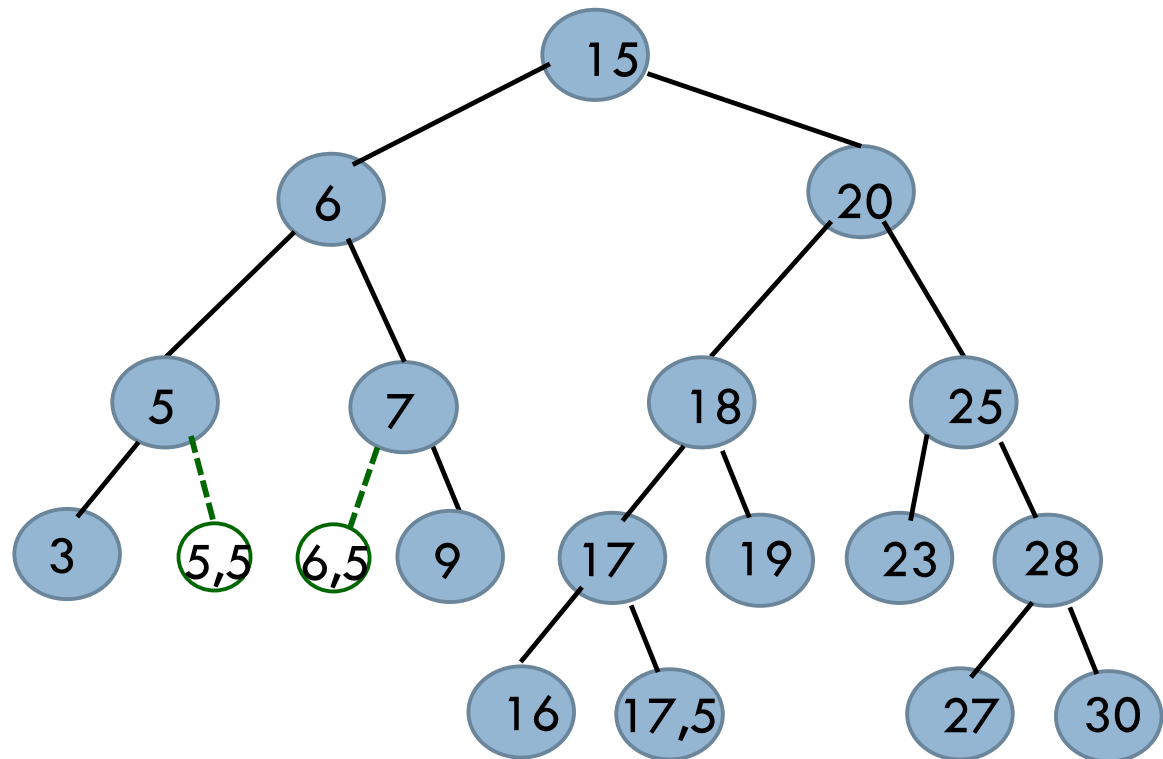
Árvore AVL

□ Rotação dupla



Árvore AVL

□ Rotação dupla



Exercícios

1. Implemente uma árvore de pesquisa binária com as funcionalidades apresentadas nesta aula
2. Modifique sua implementação para que sua Árvore seja uma Árvore AVL

Referências

1. Introduction to Algorithms: A Creative Approach, Udi Manber
2. Introduction to Algorithms, Third Edition, Thomas H. Cormen
3. Slides: Prof^a Roseli Ap. Francelin Romero, USP

